ONE8 FOOTWEAR E-COMMERCE DATABASE

DBMS Task: Sales and Customer Analytics System

Objective: Apply COUNT(), SUM(), AVG(), MIN() and MAX(); generate total sales reports; find top customers by purchase amount; identify best-selling products; and perform category-wise sales analysis.

Tables Used

Table	Purpose
Customers	Customer name, email and city.
Products	ONE8 footwear product, category and price.
Orders	Customer order date and status.
Order_Details	Products, quantities and unit prices per order.
Payments	Payment method, amount and status.

Aggregate Functions

COUNT() counts records. **SUM()** adds numeric values. **AVG()** calculates the average. **MIN()** returns the smallest value. **MAX()** returns the largest value.

1. Apply COUNT(), SUM(), AVG(), MIN() and MAX()

```
SELECT COUNT(*) AS Total_Orders FROM Orders;

SELECT SUM(Quantity) AS Units_Sold,
       SUM(Quantity*Unit_Price) AS Total_Sales
FROM Order_Details;

SELECT AVG(Price) AS Average_Price,
       MIN(Price) AS Minimum_Price,
       MAX(Price) AS Maximum_Price
FROM Products;
```

These queries count orders and calculate units sold, total sales and product price statistics. The SQL file also calculates minimum, maximum and average order values.

2. Generate Total Sales Reports

```
SELECT COUNT(DISTINCT Order_ID) AS Total_Orders,
       SUM(Quantity) AS Units_Sold,
       SUM(Quantity*Unit_Price) AS Total_Sales
FROM Order_Details;

SELECT YEAR(o.Order_Date) AS Sales_Year,
       MONTH(o.Order_Date) AS Sales_Month,
       SUM(od.Quantity*od.Unit_Price) AS Monthly_Sales
FROM Orders o
JOIN Order_Details od ON o.Order_ID=od.Order_ID
GROUP BY YEAR(o.Order_Date),MONTH(o.Order_Date);
```

The reports calculate overall sales, total units sold, number of orders and monthly sales performance.

3. Find Top Customers Based on Purchase Amount

```
SELECT c.Customer_ID,c.Customer_Name,
       COUNT(DISTINCT o.Order_ID) AS Total_Orders,
       SUM(od.Quantity*od.Unit_Price) AS Purchase_Amount
FROM Customers c
JOIN Orders o ON c.Customer_ID=o.Customer_ID
JOIN Order_Details od ON o.Order_ID=od.Order_ID
GROUP BY c.Customer_ID,c.Customer_Name
ORDER BY Purchase_Amount DESC LIMIT 5;
```

Customers are ranked from highest to lowest based on their total purchase amount. LIMIT 5 displays the top five customers.

4. Identify Best-Selling Products

```
SELECT p.Product_Name,p.Category,
       SUM(od.Quantity) AS Quantity_Sold,
       SUM(od.Quantity*od.Unit_Price) AS Total_Revenue
FROM Products p
JOIN Order_Details od ON p.Product_ID=od.Product_ID
GROUP BY p.Product_ID,p.Product_Name,p.Category
ORDER BY Quantity_Sold DESC,Total_Revenue DESC;
```

The products are sorted by quantity sold. Revenue is used as a secondary sorting condition.

5. Perform Category-Wise Sales Analysis

```
SELECT p.Category,
       COUNT(DISTINCT od.Order_ID) AS Total_Orders,
       SUM(od.Quantity) AS Units_Sold,
       SUM(od.Quantity*od.Unit_Price) AS Category_Sales,
       AVG(od.Unit_Price) AS Average_Selling_Price
FROM Products p
JOIN Order_Details od ON p.Product_ID=od.Product_ID
GROUP BY p.Category
ORDER BY Category_Sales DESC;
```

This report compares categories by number of orders, units sold, sales amount and average selling price. The SQL file also includes a category sales percentage report and city-wise sales report.

How to Run

1. Open MySQL Workbench and connect to your MySQL server.
2. Open ONE8_Sales_Customer_Analytics.sql.
3. Review the DROP TABLE statements before execution. They recreate the sample tables and remove existing data in those tables.
4. Execute the script to create the tables, insert sample ONE8 data and run the analytics queries.

Conclusion

The Sales and Customer Analytics System uses SQL aggregate functions and grouped reports to analyse sales performance, customer spending, product popularity and category-wise revenue. These results help an e-commerce business understand its performance and make informed decisions.